

Шпаргалка для защиты — QR-разложение на OpenMP

Цифры от запуска к запуску немного «плавают» (зависят от загрузки ПК) — это нормально, программа усредняет по 3 прогонам. Главное — закономерности.

1. О чём проект (одно предложение)

Реализовал QR-разложение матрицы методом Хаусхолдера, распараллелил тяжёлую часть на OpenMP, замерил ускорение на матрицах разного размера и сравнил с теоретическим пределом по закону Амдала.

2. Формулы (выучить)

- **QR-разложение:** $A = Q \cdot R$, где Q — ортогональная ($Q^T \cdot Q = I$), R — верхняя треугольная.
- **Отражение Хаусхолдера:** $H = I - 2 \cdot v \cdot v^T / (v^T \cdot v)$ — обнуляет элементы под диагональю.
- **Ускорение:** $S = T(1 \text{ поток}) / T(N \text{ потоков})$
- **Эффективность:** $E = S / N \cdot 100\%$
- **Закон Амдала:** $S(N) = 1 / ((1 - p) + p/N)$, где p — доля параллельного кода (0.82)
- **Предел:** $S_{\text{max}} = 1 / (1 - p) = 1 / 0.18 = 5.56x$

3. Ключевые цифры (мой прогон, 12 потоков)

Матрица	Лучшее ускорение	Потоков
64×64	~1.0x (не окупается)	1
128×128	~2.4x	4
256×256	~4.3x	8
512×512	~4.7x (лучший результат)	8
1024×1024	~2.5x	16
2048×2048	~3.2x (база ~35 секунд!)	16

Потолок Амдала: **5.56x**. Все реальные значения ниже — теория соблюдается.

4. Что говорить по каждому графику

- **chart_amdahl.png** — кривые ускорения для разных p ; жирная линия $p=0.82$ — наша. Каждая выходит на свой потолок $1/(1-p)$. Красные точки — реальные замеры, ложатся около нашей кривой → программа ведёт себя как предсказывает теория.
- **chart_speedup.png** — по каждому размеру: красная линия (реальное S), синяя (предел Амдала), оранжевая зона (потери на накладные расходы). На мелких матрицах красная низкая, на 512×512 — самая высокая.
- **chart_efficiency.png** — тепловая карта. Эффективность **падает вправо** (больше потоков → больше накладных расходов) и **растёт вниз** (большие матрицы лучше окупают распараллеливание). Зелёное = хорошо.
- **chart_time.png** — время от числа потоков (лог и линейная шкала). Виден обвал времени для 2048: с ~35 с (1 поток) до ~11 с (16 потоков).

5. Как написана программа

Функция	Что делает
<code>make_random()</code>	случайная матрица (seed=42 → воспроизводимость)
<code>qr_sequential()</code>	разложение в 1 поток — эталон
<code>qr_parallel()</code>	то же + <code>#pragma omp parallel for</code> на внутренних циклах
<code>bench()</code>	замер времени, среднее по 3 прогонам
<code>main()</code>	гоняет все размеры × потоки, считает метрики, пишет CSV и графики

Где параллелизм (главный вопрос!): два внутренних цикла в `qr_parallel` — применение отражения к столбцам R и строкам Q . Их итерации независимы → раздаём по потокам через `#pragma omp parallel for`

`schedule(static)` . **Внешний цикл по столбцам — последовательный** (каждый шаг зависит от предыдущего), это и есть 18% непараллелизуемого кода.

6. Вопросы и ответы

Почему ускорение не равно числу потоков? Закон Амдала: 18% кода последовательны + накладные расходы на потоки.

Почему на 16 потоках бывает медленнее, чем на 8? В машине 12 аппаратных потоков. 16 конкурируют за ядра + расходы на синхронизацию растут.

Что такое *superlinear* / эффективность > 100%? Кэш-эффект: при делении данных кусок каждого потока лучше влезает в кэш CPU, обращений к медленной памяти меньше. Не нарушает Амдала — он не учитывает кэш.

Откуда $p = 0.82$? Это оценка доли параллельного кода для теоретической кривой. Проверяется по формуле Карпа–Флэтта: $e = (1/S - 1/N)/(1 - 1/N)$, $p = 1 - e$. Для больших матриц выходит ~0.73–0.88 — 0.82 попадает в диапазон.

Зависит ли p только от железа? Нет. Алгоритмическая доля задаётся самим алгоритмом и размером матрицы (от железа не зависит). А реально измеренное p дополнительно зависит от числа потоков и накладных расходов машины — поэтому слегка плавает.

Почему матрицы случайные, а не реальные? Для замера параллелизма важен размер и плотность матрицы, а не происхождение: время QR зависит только от n . Seed фиксирован → воспроизводимость. Чтение формата .mtx добавляется одной функцией, если нужно.

7. Порядок показа

- Открыть `qroperm.slnx` в Visual Studio.
- Выбрать **Release | x64**.
- **Ctrl+F5** → подождать (на 2048 ~1–2 мин).
- Показать таблицу в консоли → сводку → потолок 5.56x.
- Открыть `benchmark_results.csv` (в Excel) и 4 файла `chart_*.png`.